ASSIGNMENT 3

Database Design Analysis

Project: Airline Reservation System | Course: Database Management Systems

Objectives covered

- Identify data redundancy and insertion, deletion and update anomalies in the design
- Identify the functional dependencies between attributes
- Explain how poor database design leads to these anomalies
- Propose improvements to the design based on the analysis

1. Introduction and Approach

The Airline Reservation System stores information about airlines, aircraft, airports, flights, passengers, bookings, tickets and payments. A common first attempt at designing such a system is to record every reservation in **one wide table**, because that mirrors the paper or spreadsheet a booking clerk would use. This assignment analyses that initial design (called **RESERVATION_FLAT**) to show exactly where redundancy and anomalies appear, lists the functional dependencies that cause them, and then shows how the finalized **eight-table design** removes them.

All figures and query results in this report were produced by running SQL against the project database (SQLite 3.45) loaded with the sample data from Assignment 7: 25 passengers, 20 flights, 20 bookings and 30 tickets. The flat table was built from that data with a join, so the redundancy counts are real, not estimates.

2. The Initial Design: RESERVATION_FLAT

In the initial design one row is stored for every ticket, and each row repeats everything known about the booking, the passenger, the flight, the aircraft, the airline and the two airports. The table has 26 columns:

Initial (un-normalized) relation

```
RESERVATION_FLAT (
    booking_ref, booking_date, booking_status, booked_by,
    passport_no, passenger_name, email, phone,
    flight_number, departure_time, arrival_time, base_fare, flight_status,
    aircraft_reg, aircraft_model, total_seats,
    airline_name, iata_code,
    origin_code, origin_city, dest_code, dest_city,
    seat_no, travel_class, fare, ticket_status )

Chosen key : (booking_ref, passport_no, flight_number, departure_time)
```

A sample of the data, as it appears in the flat table, makes the repetition visible (selected columns shown):

SELECT ... FROM reservation_flat

```
+---------+-----------------+----------------+--------+-----------+----------------+-----------+------+-------+
| booking | passenger_name  | phone          | flight | airline   | aircraft       | origin    | seat | fare  |
+---------+-----------------+----------------+--------+-----------+----------------+-----------+------+-------+
| 75HZS2  | Aarav Sharma    | +91-6399869879 | 6E-501 | IndiGo    | Airbus A320neo | Bengaluru | 12A  |  5000 |
| D5BUXU  | Priya Nair      | +91-6141707536 | AI-807 | Air India | Airbus A320    | Bengaluru | 14C  |  4510 |
| D5BUXU  | Rohan Mehta     | +91-9815188447 | AI-807 | Air India | Airbus A320    | Bengaluru | 14D  |  4510 |
| YVB5JY  | Vikram Singh    | +91-9684697880 | 6E-501 | IndiGo    | Airbus A320neo | Bengaluru | 2A   | 16020 |
| BXNDHL  | Rohan Mehta     | +91-9815188447 | 6E-501 | IndiGo    | Airbus A320neo | Bengaluru | 8A   |  4680 |
| BXNDHL  | Ananya Iyer     | +91-6557616987 | 6E-501 | IndiGo    | Airbus A320neo | Bengaluru | 8B   |  4680 |
| BBT84S  | Mohammed Faizan | +91-9804571784 | AI-807 | Air India | Airbus A320    | Bengaluru | 10A  |  5160 |
+---------+-----------------+----------------+--------+-----------+----------------+-----------+------+-------+
7 rows in set
```

The airline name 'IndiGo', the aircraft model, and the origin city 'Bengaluru' are typed again on every ticket row of the flight.

3. Data Redundancy

Data redundancy means the same fact is stored in more than one place. In RESERVATION_FLAT every fact about a flight, passenger, aircraft or airport is stored once per ticket instead of once in total. The query below counts, for each group of attributes, how many rows store it and how many distinct facts exist.

Redundancy measurement

```
SELECT 'Passenger details' AS fact_group, COUNT(*) AS rows_storing_it,
  (SELECT COUNT(*) FROM (SELECT DISTINCT passport_no, passenger_name, email, phone
                         FROM reservation_flat)) AS distinct_facts
FROM reservation_flat
UNION ALL
SELECT 'Flight details', COUNT(*),
  (SELECT COUNT(*) FROM (SELECT DISTINCT flight_number, departure_time, arrival_time,
                         base_fare, flight_status FROM reservation_flat))
FROM reservation_flat
UNION ALL
SELECT 'Aircraft details', COUNT(*),
  (SELECT COUNT(*) FROM (SELECT DISTINCT aircraft_reg, aircraft_model, total_seats
                         FROM reservation_flat))
FROM reservation_flat
UNION ALL
SELECT 'Airline details', COUNT(*),
  (SELECT COUNT(*) FROM (SELECT DISTINCT airline_name, iata_code FROM reservation_flat))
FROM reservation_flat
UNION ALL
SELECT 'Origin airport details', COUNT(*),
  (SELECT COUNT(*) FROM (SELECT DISTINCT origin_code, origin_city FROM reservation_flat))
FROM reservation_flat
UNION ALL
SELECT 'Booking details', COUNT(*),
  (SELECT COUNT(*) FROM (SELECT DISTINCT booking_ref, booking_date, booking_status,
                         booked_by FROM reservation_flat))
FROM reservation_flat
```

OUTPUT

```
+------------------------+-----------------+----------------+
| fact_group             | rows_storing_it | distinct_facts |
+------------------------+-----------------+----------------+
| Passenger details      |              30 |             23 |
| Flight details         |              30 |             20 |
| Aircraft details       |              30 |             10 |
| Airline details        |              30 |              6 |
| Origin airport details |              30 |              6 |
| Booking details        |              30 |             20 |
+------------------------+-----------------+----------------+
6 rows in set
```

Fact group	Stored (rows)	Needed (distinct)	Redundant copies	Redundancy
Passenger details	30	23	7	23%
Flight details	30	20	10	33%
Aircraft details	30	10	20	67%
Airline details	30	6	24	80%
Origin airport details	30	6	24	80%
Booking details	30	20	10	33%

Between a quarter and four-fifths of the stored values are copies (the more entities repeat across tickets, the higher the share). Apart from wasting storage, each copy is another place where the value can drift out of step with the others. The redundancy has three root causes: (i) one relation describes several different entities (booking, passenger, flight, aircraft, airline, airport); (ii) attributes that describe an entity are stored next to the key of a different entity; and (iii) a payment can occur several times for one booking, which would be a repeating group inside the row if payments were added to this table.

4. Anomalies in the Initial Design

Each anomaly below was reproduced on the real RESERVATION_FLAT table. Each experiment is undone afterwards (by a ROLLBACK or by deleting the test row), so the data is unchanged.

4.1 Insertion anomaly

An insertion anomaly occurs when a new fact cannot be recorded unless unrelated data is recorded with it. The key of RESERVATION_FLAT contains **booking_ref, passport_no, flight_number and departure_time**, all of which must be non-NULL. Therefore:

- A new airport (for example **GOI - Goa**) cannot be stored until a ticket is sold on a flight that uses it.
- A new aircraft that has been purchased (registration VT-SPQ) cannot be stored until it is assigned to a flight with a passenger.
- A newly registered passenger who has not yet bought a ticket cannot be saved, and a newly scheduled flight cannot be published until the first ticket is sold.
- To store such facts anyway, the designer must invent dummy values for booking_ref or passport_no, which pollutes the data and breaks reports.

Insertion anomaly attempt

```
-- Try to record the new airport GOI (Goa) in the flat table.
-- No ticket exists yet, so the key columns would have to be filled with dummies.
INSERT INTO reservation_flat (origin_code, origin_city)
VALUES ('GOI', 'Goa');
-- The design forces a choice:  (a) refuse the airport   or   (b) insert a fake ticket row.
```

Result: the airport only 'fits' as a half-empty ghost row

```
+-------------+-------------+---------------+-------------+-------------+------+
| booking_ref | passport_no | flight_number | origin_code | origin_city | fare |
+-------------+-------------+---------------+-------------+-------------+------+
| NULL        | NULL        | NULL          | GOI         | Goa         | NULL |
+-------------+-------------+---------------+-------------+-------------+------+
1 row in set
```

In a flat table with a declared primary key the insert is rejected outright; without one (as here) it is accepted only as a meaningless row with NULL booking, passenger and fare. Either outcome is an anomaly.

4.2 Deletion anomaly

A deletion anomaly occurs when removing one fact accidentally destroys another. Flight **6E-319 (Bengaluru to Kochi)** has only one ticket. If that passenger cancels and the ticket row is deleted, all knowledge of the flight, the Kochi airport and its aircraft disappears.

SQL

```
SELECT DISTINCT flight_number, origin_city, dest_city, aircraft_model, airline_name
FROM reservation_flat WHERE dest_city = 'Kochi';          -- before
```

BEFORE

```
+---------------+-------------+-----------+----------------+--------------+
| flight_number | origin_city | dest_city | aircraft_model | airline_name |
+---------------+-------------+-----------+----------------+--------------+
| 6E-319        | Bengaluru   | Kochi     | Airbus A321neo | IndiGo       |
+---------------+-------------+-----------+----------------+--------------+
1 row in set
```

SQL

```
DELETE FROM reservation_flat WHERE flight_number = '6E-319';   -- the only ticket is cancelled
SELECT DISTINCT flight_number, origin_city, dest_city, aircraft_model, airline_name
FROM reservation_flat WHERE dest_city = 'Kochi';          -- after
```

AFTER (Kochi, the flight and its schedule are gone)

```
Empty set (0 rows)
```

4.3 Update (modification) anomaly

An update anomaly occurs when a single real-world change must be applied to many rows. If it is applied to only some of them, the table contradicts itself. Passenger **Aarav Sharma** appears on several rows (3 tickets). He changes his phone number, but the update is applied to only one row (for example by a clerk who edits the ticket he is looking at):

```sql
UPDATE reservation_flat SET phone = '+91-9000000000'
WHERE passenger_name = 'Aarav Sharma' AND flight_number = '6E-501';   -- only ONE ticket row edited

SELECT DISTINCT passenger_name, phone FROM reservation_flat
WHERE passenger_name = 'Aarav Sharma';
```

Result: two different phone numbers for the same person

```
+----------------+----------------+
| passenger_name | phone          |
+----------------+----------------+
| Aarav Sharma   | +91-9000000000 |
| Aarav Sharma   | +91-6399869879 |
+----------------+----------------+
2 rows in set
```

Similar update anomalies exist for every repeated group: renaming an airport city, changing the seating capacity of an aircraft, or revising a flight's arrival time would need to touch every ticket row of that flight, and a single missed row leaves the database inconsistent.

4.4 Summary of anomalies

Anomaly	Example in RESERVATION_FLAT	Cause
Insertion	Cannot store airport GOI, aircraft VT-SPQ, or a passenger or flight with no ticket yet	Key needs ticket data; unrelated entities share one row
Deletion	Cancelling the only ticket of 6E-319 erases the flight, aircraft and Kochi airport	Facts about different entities are stored only inside ticket rows
Update	A phone number or airport city is stored per ticket; partial updates leave contradictions	Non-key attributes repeated (determined by only part of the key)

5. Functional Dependencies

A **functional dependency (FD)** X → Y means that for each value of X there is exactly one value of Y. FDs are derived from the business rules of an airline reservation system (for example: a passport number identifies one traveller; a flight number on a given departure time identifies one scheduled flight; an aircraft registration identifies one physical aircraft).

#	Functional dependency	Meaning (business rule)	Type w.r.t. the key
FD1	`booking_ref → booking_date, booking_status, booked_by`	A booking reference identifies one booking and who made it	Partial
FD2	`passport_no → passenger_name, email, phone`	A passport number identifies one traveller	Partial
FD3	`email → passport_no`	E-mail is unique per passenger (candidate key)	Candidate-key FD
FD4	`(flight_number, departure_time) → arrival_time, base_fare, flight_status, aircraft_reg, origin_code, dest_code`	A scheduled flight has one timing, fare, status, aircraft and route	Partial
FD5	`aircraft_reg → aircraft_model, total_seats, airline_name`	A registration identifies one aircraft owned by one airline	Transitive (via FD4)
FD6	`airline_name → iata_code and iata_code → airline_name`	Name and IATA code identify each other	Transitive (via FD4, FD5)
FD7	`origin_code → origin_city; dest_code → dest_city`	An airport code identifies one city	Transitive (via FD4)

#	Functional dependency	Meaning (business rule)	Type w.r.t. the key
FD8	`(booking_ref, passport_no, flight_number, departure_time) → seat_no, travel_class, fare, ticket_status`	A ticket belongs to one booking, traveller and flight and has one seat, class and fare	Full dependency on the key
FD9	`(flight_number, departure_time, seat_no) → passport_no`	A seat on a flight is occupied by one traveller (for non-cancelled tickets)	Alternate candidate key

Normal-form diagnosis of RESERVATION_FLAT. The relation is in 1NF (all values atomic, once payments are moved out as a repeating group). It **violates 2NF** because FD1, FD2 and FD4 are *partial dependencies*: non-key attributes depend on only a part of the composite key. It **violates 3NF** because FD5, FD6 and FD7 are *transitive dependencies*: a non-key attribute determines other non-key attributes (for example flight → aircraft_reg → aircraft_model).

6. How Poor Design Leads to These Anomalies

Anomalies are not random bugs; each one is the direct, predictable consequence of a particular kind of functional dependency that has been left inside a relation whose key is something else.

Design flaw	Dependency involved	Resulting problem
Several entities mixed in one table	Partial dependencies FD1, FD2, FD4	Facts such as a passenger's phone are stored once per ticket (redundancy). A change must be repeated in many rows (**update anomaly**).
Entity facts live only inside child rows	FD2, FD4, FD5, FD7 (determinant is not the key)	An entity exists only while at least one ticket row exists. Deleting the last ticket deletes the entity (**deletion anomaly**).
Key made of attributes from unrelated entities	Composite key (booking, passport, flight, departure)	All key parts are NOT NULL, so a lone airport, aircraft, passenger or flight cannot be stored (**insertion anomaly**).
Transitive chains	flight → aircraft → airline; flight → airport → city	Airline or city names are repeated for every flight and ticket; renaming one value touches thousands of rows.
Repeating groups	One booking has many payment attempts	A single row cannot hold them without duplicating the booking or using list-like columns (violates 1NF).

In addition, redundancy makes queries and storage more expensive, makes integrity rules harder to enforce (nothing stops two different spellings of the same airline), and makes the system fragile when requirements change. The cure is to decompose the relation so that **every determinant is a key** of its own relation, which is exactly what normalization does.

7. Improvement: Normalization of the Design

7.1 First Normal Form (1NF)

Atomic columns are ensured (names are split into first_name and last_name; each seat is one value). The repeating group *payments* is removed from the reservation row and stored in its own relation PAYMENT(payment_id, booking_id, …), one row per payment attempt.

7.2 Second Normal Form (2NF): remove partial dependencies

Every attribute that depends on only part of the composite key moves to a relation keyed by that part:

```
After 2NF
BOOKING   (booking_ref, booking_date, booking_status, booked_by)                    <- FD1
PASSENGER (passport_no, name, email, phone, date_of_birth, nationality)              <- FD2
FLIGHT    (flight_number, departure_time, arrival_time, base_fare, flight_status,
           aircraft_reg, aircraft_model, total_seats, airline_name, iata_code,
           origin_code, origin_city, dest_code, dest_city)                             <- FD4
TICKET    (booking_ref, passport_no, flight_number, departure_time,
           seat_no, travel_class, fare, ticket_status)                                 <- FD8
```

7.3 Third Normal Form (3NF): remove transitive dependencies

FLIGHT still contains aircraft, airline and airport facts that depend on non-key attributes (FD5, FD6, FD7), so these are extracted:

```
After 3NF
AIRLINE  (airline_id PK, airline_name, iata_code, country)                      <- FD6
AIRPORT  (airport_code PK, airport_name, city, country)                              <- FD7
AIRCRAFT (aircraft_id PK, registration_no, model, total_seats, status, airline_id FK) <- FD5
FLIGHT   (flight_id PK, flight_number, aircraft_id FK, origin_code FK, destination_code FK,
          departure_time, arrival_time, base_fare, status)
```

Surrogate keys. Long composite natural keys are replaced by short integer surrogate keys (*flight_id*, *passenger_id*, *ticket_id* ...), with the natural identifiers kept as **UNIQUE** constraints (*passport_no*, *email*, *booking_ref*, *(flight_number, departure_time)*). Note that the airline of a flight is not stored in FLIGHT; it is reached through *aircraft.airline_id*, because storing it in both places would recreate the transitive dependency flight → aircraft → airline.

7.4 The finalized relational design

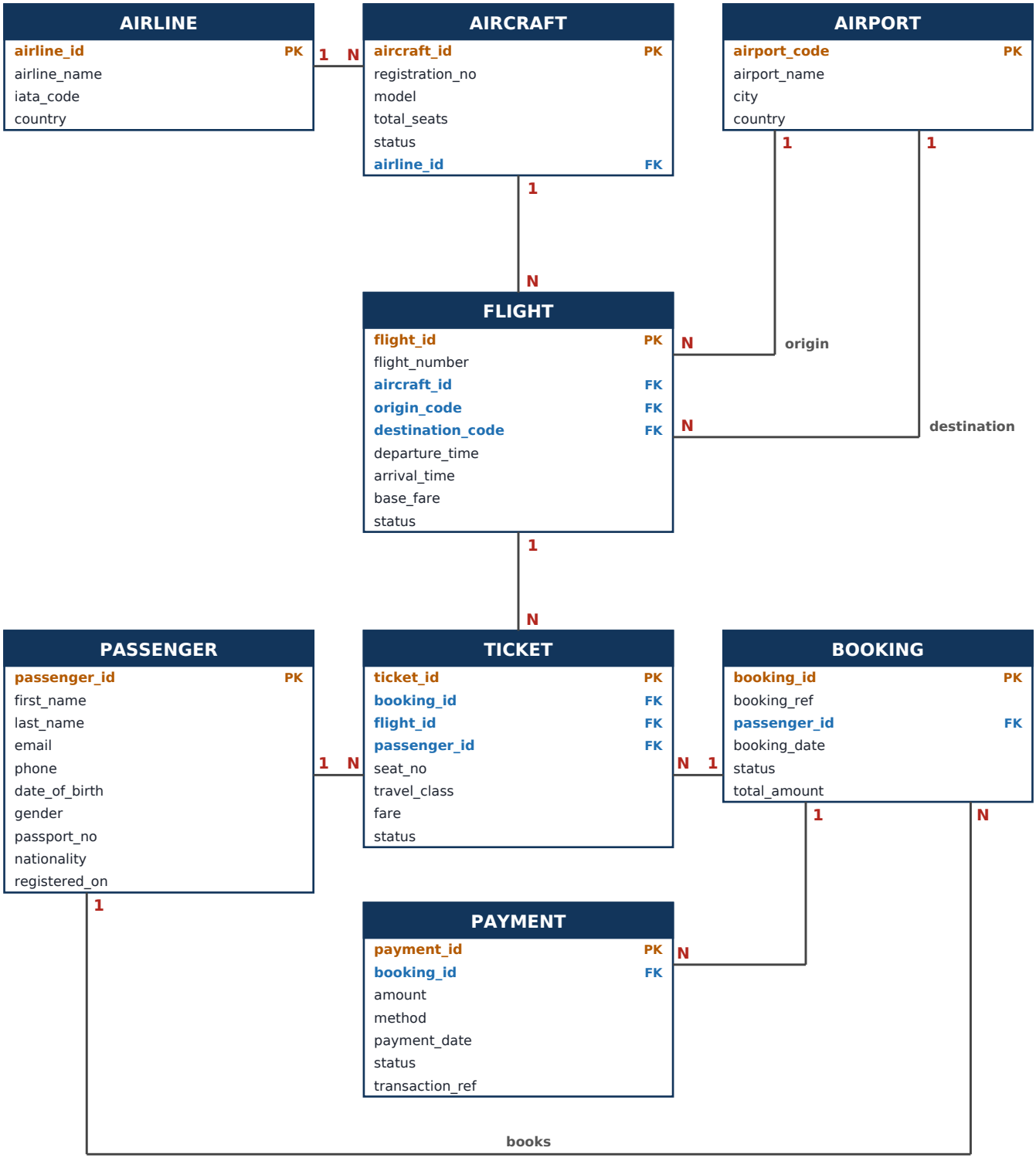


Figure 1: ER / relational diagram of the finalized 3NF design (PK = primary key, FK = foreign key, 1 : N = one-to-many).

Relation	Primary key	Foreign keys	Determinants (FDs)	Normal form
airline	airline_id	–	airline_id → all; iata_code ↔ airline_name (candidate keys)	BCNF
airport	airport_code	–	airport_code → name, city, country	BCNF
aircraft	aircraft_id	airline_id → airline	aircraft_id → all; registration_no → all (candidate key)	BCNF
flight	flight_id	aircraft_id → aircraft; origin_code, destination_code → airport	flight_id → all; (flight_number, departure_time) → all (candidate key)	BCNF
passenger	passenger_id	–	passenger_id → all; passport_no, email → all (candidate keys)	BCNF
booking	booking_id	passenger_id → passenger	booking_id → all; booking_ref → all (candidate key)	3NF / BCNF*
ticket	ticket_id	booking_id, flight_id, passenger_id	ticket_id → all; (flight_id, seat_no) → all (candidate key)	BCNF
payment	payment_id	booking_id → booking	payment_id → all; transaction_ref → all (candidate key)	BCNF

**booking.total_amount is a deliberately kept derived value; see Section 9. Every determinant above is a candidate key, so each relation is in BCNF (and hence also 3NF).*

8. Re-testing the Anomalies on the Improved Design

The same three experiments are repeated on the normalized tables. All three succeed without side effects.

Insertion: store a new airport and a new aircraft with no tickets

```sql
INSERT INTO airport (airport_code, airport_name, city, country)
VALUES ('PNQ', 'Pune International Airport', 'Pune', 'India');
INSERT INTO aircraft (aircraft_id, registration_no, model, total_seats, status, airline_id)
VALUES (12, 'VT-QPA', 'Boeing 737-8 MAX', 189, 'Active', 4);
SELECT airport_code, city FROM airport WHERE airport_code = 'PNQ';
```

Result: stored independently, no ticket required

```
+--------------+------+
| airport_code | city |
+--------------+------+
| PNQ          | Pune |
+--------------+------+
1 row in set
```

Deletion: cancel the only ticket of flight 6E-319; the flight survives

```sql
DELETE FROM ticket WHERE flight_id = 16;       -- the only ticket of flight 6E-319
SELECT f.flight_number, o.city AS origin, d.city AS destination
FROM flight f JOIN airport o ON f.origin_code = o.airport_code
              JOIN airport d ON f.destination_code = d.airport_code
WHERE f.flight_id = 16;
```

Result: flight, airports and aircraft are unaffected

```
+---------------+-----------+-------------+
| flight_number | origin    | destination |
+---------------+-----------+-------------+
| 6E-319        | Bengaluru | Kochi       |
+---------------+-----------+-------------+
1 row in set
```

Update: change a phone number once

```sql
UPDATE passenger SET phone = '+91-9000000000' WHERE passenger_id = 1;   -- ONE row changed

SELECT p.first_name, p.last_name, p.phone, COUNT(t.ticket_id) AS tickets
FROM passenger p JOIN ticket t ON t.passenger_id = p.passenger_id
WHERE p.passenger_id = 1 GROUP BY p.passenger_id;
```

Result: one stored copy, so every ticket sees the new number

```
+------------+-----------+----------------+---------+
| first_name | last_name | phone          | tickets |
+------------+-----------+----------------+---------+
| Aarav      | Sharma    | +91-9000000000 |       3 |
+------------+-----------+----------------+---------+
1 row in set
```

9. Remaining Redundancy and Further Improvements

A well-normalized design is not necessarily free of every duplicate value. Two columns are intentionally kept and are controlled:

- **ticket.fare vs flight.base_fare.** They look similar but are different facts: base_fare is the *current list price*, whereas fare is the *price agreed at the moment of sale* (it depends on class and demand). Fare must not change when the airline later changes base_fare, so storing it is correct, not redundant.
- **booking.total_amount** is derived data (SUM of ticket fares) stored for fast reporting and payment checks. It is the only true controlled redundancy. A consistency query (Assignment 6) and a trigger or application rule keep it correct.

Further improvements recommended for production use:

Proposal	Reason	Benefit
Trigger (or generated view) that recomputes booking.total_amount whenever a ticket is inserted, updated or deleted	Removes the last manual-update risk	Prevents total_amount from drifting from the ticket fares
Separate AIRCRAFT_MODEL table (model, manufacturer, standard seat count)	The same model string is repeated for every aircraft of that type	Fewer typing errors; single place to correct a model name
Add a FLIGHT_SEAT / SEAT_MAP table (flight_id, seat_no, class)	Currently any seat string is accepted for a ticket	Enforces valid seats per aircraft and class; supports seat selection screens
Replace long CHECK (status IN ...) lists with lookup tables (status codes)	Adding a new status needs an ALTER of the constraint	Easier maintenance and multilingual labels
Add a COUNTRY table referenced by airline, airport, and passenger.nationality	Country names are free text and repeated	Standard codes (ISO), consistent spelling
Add indexes on foreign keys and on flight(origin, destination, departure)	Search and join queries scan whole tables	Faster flight search and reports (already created in Assignment 5)
Add a FARE_HISTORY / audit table	Changes to prices and ticket status are overwritten	Traceability and dispute resolution

10. Conclusion

The analysis shows that a single reservation table suffers from heavy redundancy (up to 80% duplicated values for airline and airport facts in our small sample, and rising as more tickets are sold) and from all three kinds of anomaly, because partial and transitive functional dependencies were left inside the relation. Decomposing the design into eight relations in which every determinant is a candidate key (BCNF) removed those dependencies: new facts can be stored independently, deleting a ticket no longer erases a flight, and each fact is stored in exactly one place. The finalized design therefore forms a solid base for the table creation, constraints, data and queries in the following assignments.